Optimal Design of a 3-stage Ladder LC Bandpass Filter with an Arbitrary Frequency Response

Shayan Farahvash

Executive Summary

The objective of this Capstone projects is to design a 3-stage ladder LC filter to deliver a desired frequency response subject to the following conditions:

- One band pass only. For this project we set it to be between 2300MHz to 2400MHz but it can be any arbitrary span in frequency domain.
- Multiple stop bands with nonuniform rejection (see below graph for an example.

The aim of this work is start from a dataset of an analytically derived sub-optimal solutions, use that to train a ML/AI regressor, and then use regressor to predict what would be the solution to a real world multiple stopband problems.

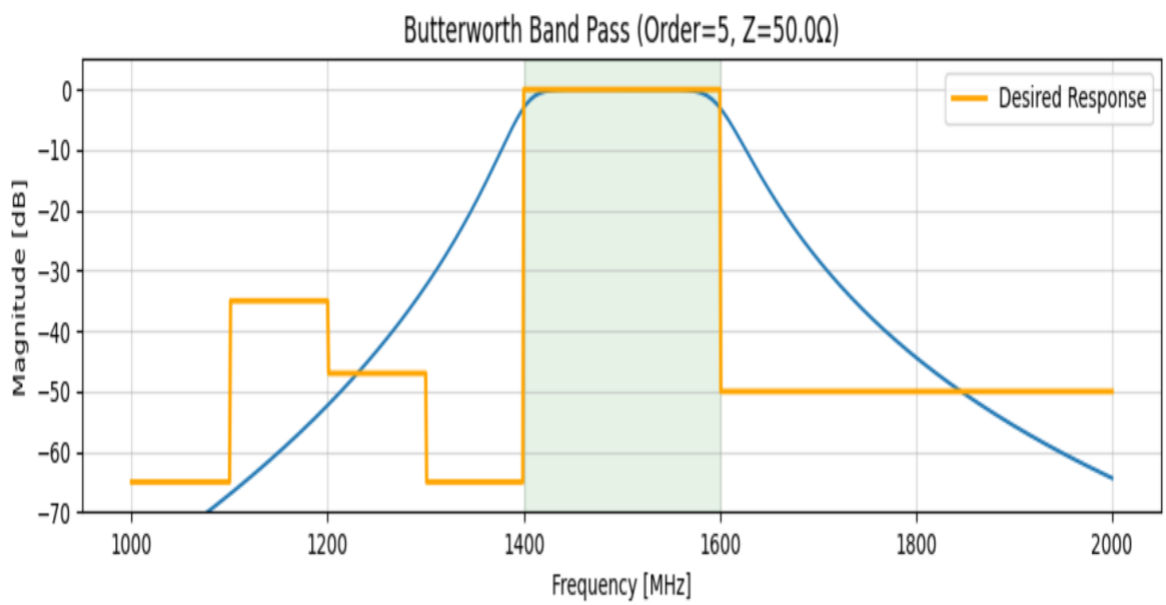

Figure 1: Desired piecewise continuous frequency response of a filter along with LC filter response. Goal: minimize the difference between the two

Rationale

Filters play a key role in wireless and wireline devices by passing a desired signal to subsequent systems while simultaneously—and most crucially—rejecting all unwanted ones. These unwanted signals (often referred to as blockers or interferers) cause a range of negative system impacts, such as desensitization, aliasing, intermodulation, and compression. The strength and frequency of such signals can vary widely, requiring a filter with a flexible rejection profile that changes as a function of frequency.

The synthesis of bandpass LC networks with a given passband is a classic problem for which semi-optimal solutions have existed for decades. However, traditional filter design techniques, although highly analytical and proven, cannot be applied to such arbitrarily shaped responses—even with the most flexible filter types (e.g., elliptical filters). While certain designs, such as the inverse Chebyshev, can provide secondary rejection levels by realizing deep notches, these levels cannot be moved around arbitrarily without compromising the filter's passband response.

Given that there is no analytical solution for the problem stated above, the current approach to filter design is often ad-hoc. Components are typically tailored by hand or using optimization algorithms, such as genetic algorithms, to provide an approximation of the desired response. This project is an attempt to apply deep learning to solve this synthesis problem.

Research Question

The objective of this project is to determine the optimal component values for a three-stage LC ladder filter to accurately approximate a target frequency response. While target specifications are often piecewise continuous, a physical LC filter is inherently constrained to a continuous response. To expand the design space, the filter topology has been augmented with additional components, enabling an AI/ML regression model to explore higher-dimensional configurations for response realization (see Figure 2).

Currently, no closed-form analytical solution exists that accounts for these additional components. The AI/ML regressor determines the values of the nine constituent components (comprising 3 inductors and 6 capacitors) to minimize the error between the desired specification and the realizable response of the hardware architecture illustrated in Figure 2.

Data Sources

The dataset was generated by randomizing component values of several three-stage Chebyshev filters, each exhibiting various pass-band ripple characteristics. During the randomization process, only those filter designs with a specified maximum ripple were retained for subsequent training. This approach ensures that the regression method is trained exclusively on high-quality designs with minimal in-band ripple. However, this alone does not guarantee an optimal design; therefore, further refinement is performed using a reinforcement learning (RL) algorithm to achieve fine-tuning.

Circuit analysis techniques are required to verify which designs exhibit acceptable frequency responses following randomization. For this purpose, the PySPice library must be installed to facilitate analysis of the filters and computation of their frequency responses. The specific methodologies for synthesizing and analyzing LC filters are outside the scope of this report, as they require specialized knowledge in circuit design and analysis. These methods are referenced in the Filter_data notebook.

Five base filters were evaluated, with frequency response samples collected at 129 points. All filters feature passbands from 2300 MHz to 2400 MHz, while frequency responses are captured over the range of 1800 MHz to 2800 MHz. Randomization of the five base filters produced 6,000 samples for training; only those passing bands with less than 3 dB attenuation were retained, and all others were excluded. Inductance values are expressed in microhenries (µH), and capacitance values in picofarads (pF). Ultimately, two dataframes are created: one containing the raw data and another with outliers removed for data cleansing purposes.

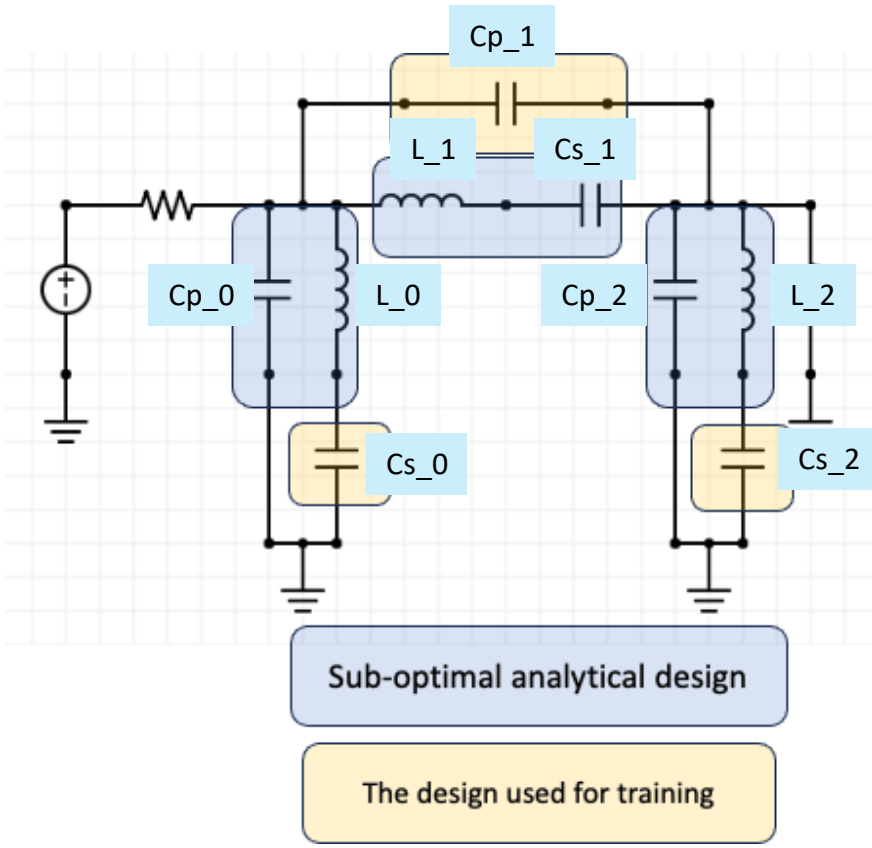

Figure 2: Topology of filter used for this project. Three components are added to expand the flexibility of design. the blue-marked components of topology are derived from analytical techniques as the starting point for randomization and simulation.

	L_0	L_1	L_2	Cs_0	Cs_1	Cs_2	Cp_0	Cp_1	Cp_2	Mag_1800.00MHz	...	Mag_2729.69MHz	Mag_2737.50MHz	Mag_2745.31MHz	Mag_2753.12MHz	Mag_2760.94MHz
0	0.1639	88.58	0.1639	1.000000e+09	51.8	1.000000e+09	27990	1	27990	-58.740883	...	-43.572800	-44.074911	-44.566316	-45.047450	-45.518722
1	0.1639	88.58	0.1639	7.115404e+06	51.8	1.252603e+07	29278	145	28177	-51.564003	...	-78.924077	-109.345099	-79.303122	-73.856148	-70.812280
2	0.1639	88.58	0.1639	1.091239e+07	51.8	1.046144e+07	28311	318	28442	-47.397272	...	-43.835646	-43.961716	-44.090729	-44.222144	-44.355494
3	0.1639	88.58	0.1639	7.473066e+06	51.8	4.111292e+06	27199	297	29078	-48.230940	...	-44.191564	-44.309525	-44.431560	-44.556974	-44.685167
4	0.1639	88.58	0.1639	7.518186e+06	51.8	1.291216e+07	28479	681	27586	-42.195290	...	-33.877630	-34.115003	-34.348387	-34.577875	-34.803559

5 rows × 138 columns

Figure 3:Dataframe structure for training

Each row in the dataframe presents the filter's frequency response (X) alongside the corresponding LC component values (y), as illustrated in Figure 3. Sample frequency responses appear in Figure 4, with the base filter highlighted in bold black.

A significant challenge in this synthesis task arises from the fact that the component values span nine orders of magnitude (refer to Figure 5). Training a neural network on such raw data leads to poor convergence and elevated Mean Absolute Percentage Error (MAPE). To address this, all component values are normalized using log10 scaling, which compresses the value range and allows the optimizer to weigh relative changes uniformly across the pF–µH spectrum.

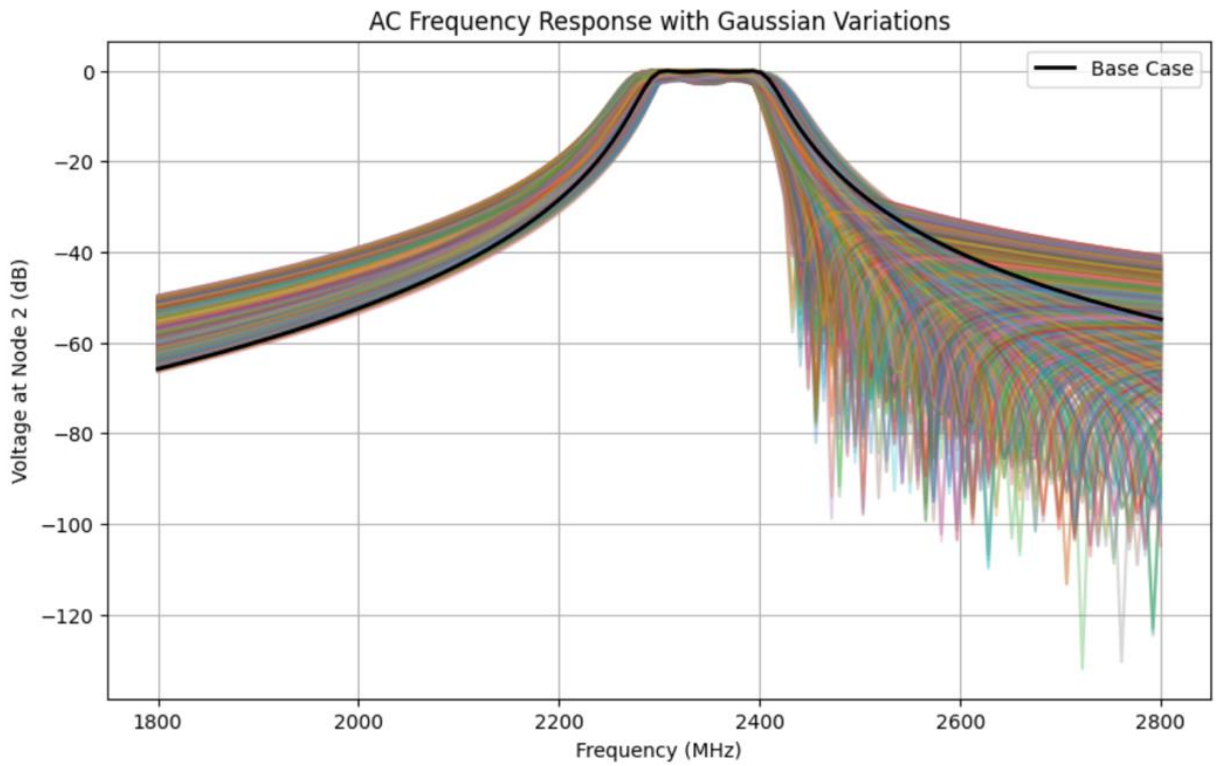

Figure 4: Samples of frequency response of filter in the training set

--- Summary Statistics of Target Variables (y_raw – Log10 Scale) ---

	L_0	L_1	L_2	Cs_0	Cs_1	Cs_2	Cp_0	Cp_1	Cp_2
count	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000	6000.000000
mean	-0.884942	1.958726	-0.884942	6.954703	1.702977	6.953824	4.547001	2.303763	4.547246
std	0.060214	0.005814	0.060214	0.192044	0.005803	0.191197	0.060194	0.457446	0.060266
min	-0.956245	1.947336	-0.956245	6.587486	1.698449	6.587483	4.429978	0.000000	4.430091
25%	-0.930332	1.960090	-0.930332	6.804491	1.699230	6.805793	4.508284	2.107210	4.509313
50%	-0.897910	1.960518	-0.897910	6.942024	1.701222	6.934866	4.559577	2.418301	4.560002
75%	-0.854804	1.962464	-0.854804	7.096977	1.701654	7.096234	4.598916	2.633468	4.598908
max	-0.785421	1.963221	-0.785421	9.000000	1.714330	9.000000	4.638579	2.844477	4.639875

Figure 5: Statistics of filter components

To assess data quality, box plots of filter components were presented in Fig6. Most components showed tight distributions, except for Cs_2 and Cs_1, which had noticeably large outliers. Outliers were detected and removed using the Interquartile Range (IQR) method—a robust approach commonly used for isolating extreme values. The process was applied to the Log10-scaled target variables (df_y), as log-scaling normalizes the wide dynamic range of component values and improves outlier detection. Values outside the interval [Q1 - 1.5 * IQR, Q3 + 1.5 * IQR]—often referred to as the "whiskers" of a box plot—were considered outliers. These outlier rows were then removed from the original, unscaled dataframe (df) to generate a new, cleaned dataset named df_cleaned.

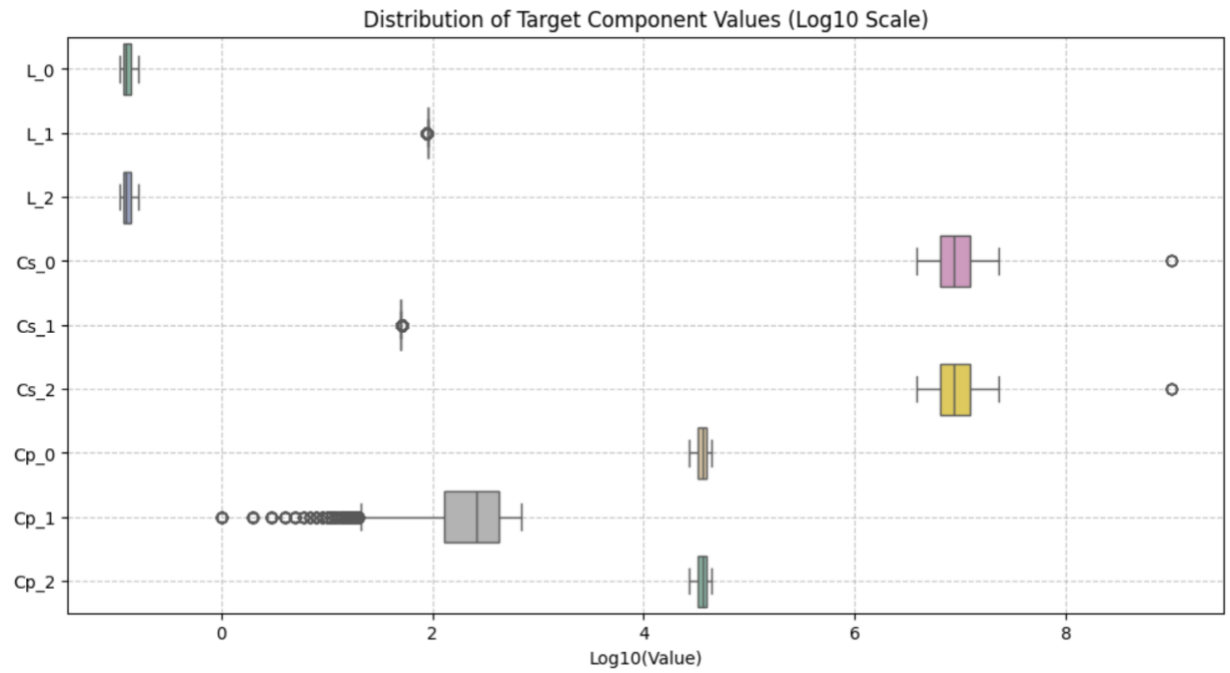

Figure 6: Boxplot of values for filter components

By analyzing the histogram of filter components (Fig. 7), additional insights can be obtained.

- Since L_0, L_1, L_2, and Cs_1 originated from the primary optimal design, their values remain unchanged as they are not necessary for achieving new frequency responses.
- Cp_0 and Cp_2 are randomized; however, their limited range suggests proximity to their optimal values.
- The process of randomization to produce a notch in the response, while maintaining low in-band insertion loss, led to considerable variation in CP_1, CS_0, and CS_2. These modifications are essential for tailoring the filter's frequency response and creating a pronounced notch at the upper edge of the passband.
- Although the filter’s frequency response was uniformly randomized, the distribution of component values does not reflect uniform or Gaussian characteristics, highlighting a nonlinear relationship between the response and the component values.

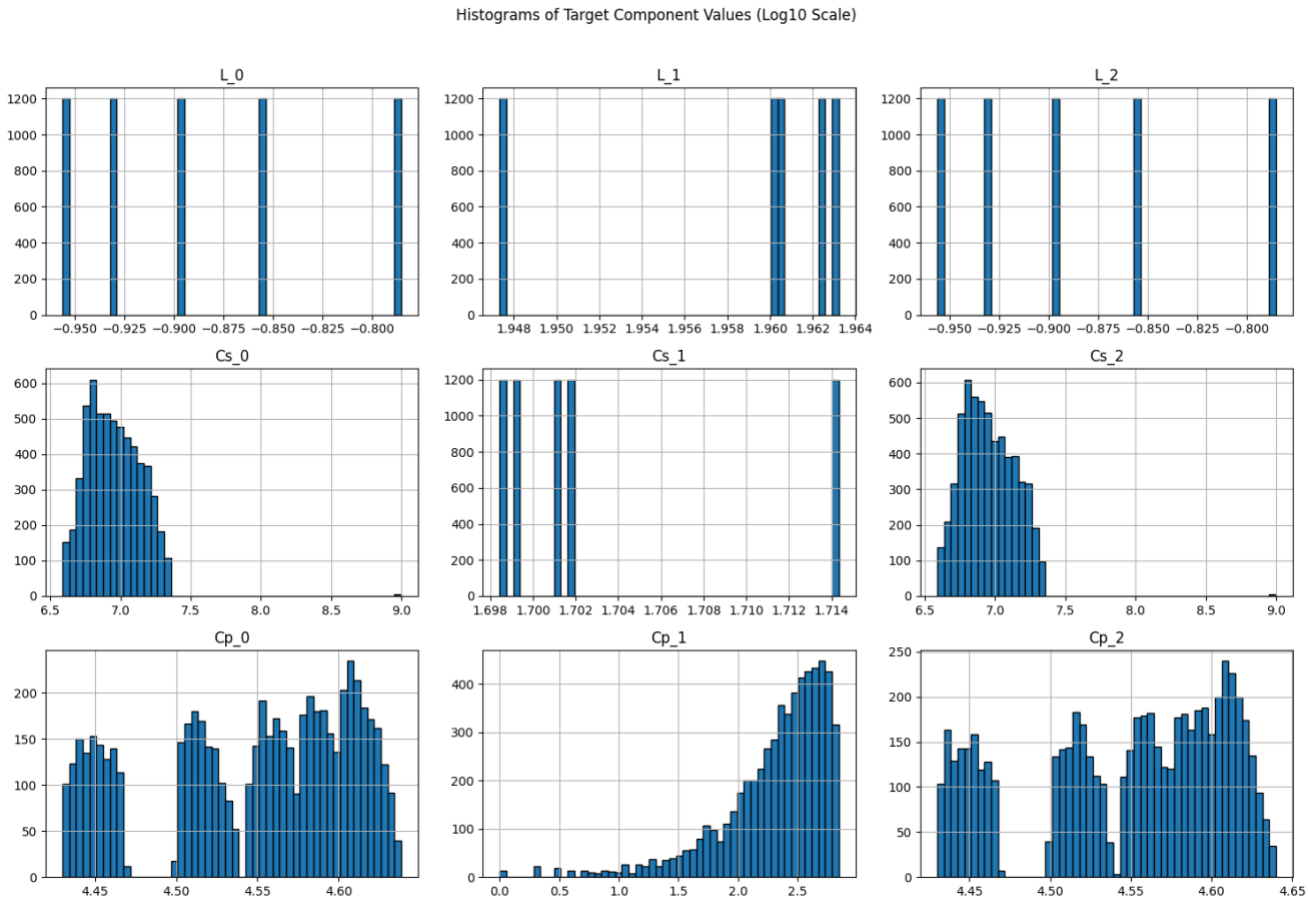

Figure 7: histogram of values for filter components

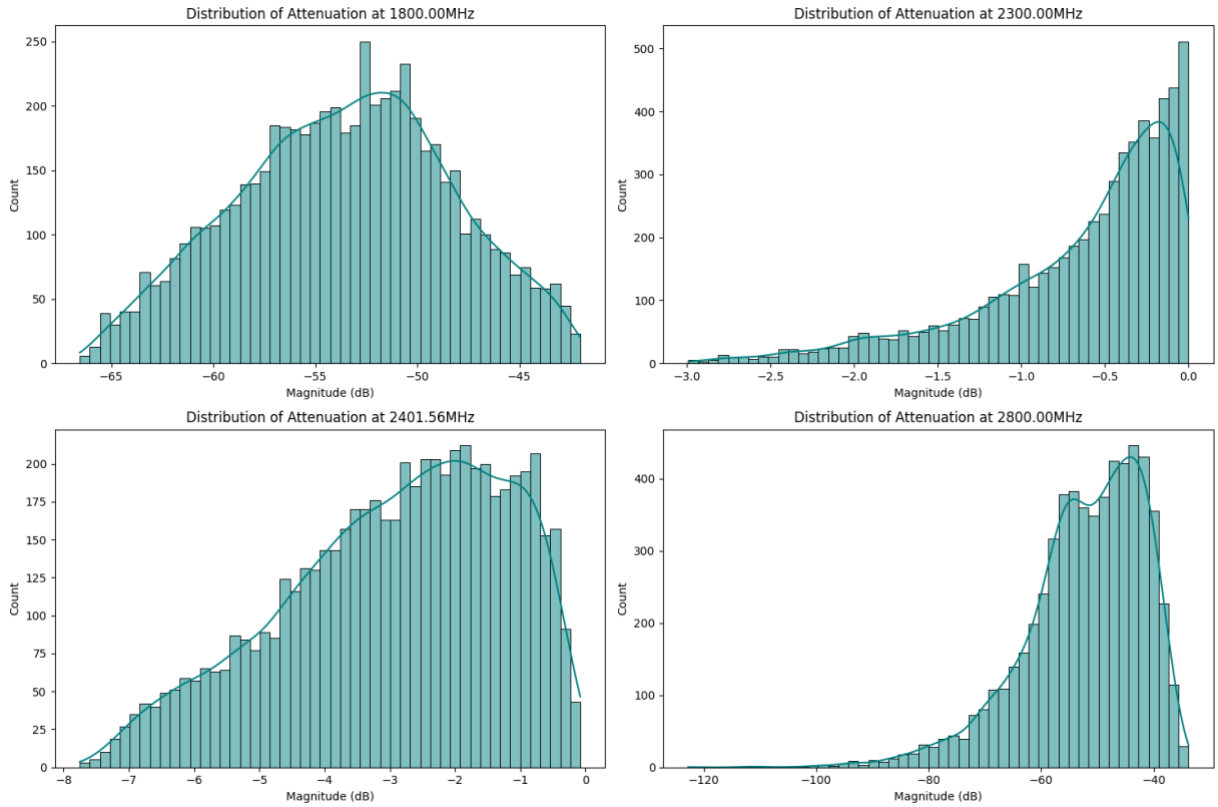

Figure 8: Histogram of filter responses at four frequencies

To determine the distribution of filter response data, five key points were selected for analysis: 1800 MHz, 2300 MHz, 2400 MHz, and 2800 MHz. The response distributions are consistent and well-formed (see Fig. 8).

- The notch frequency should not be positioned too close to the upper band edge at 2400 MHz; the bar diagram indicates that the minimum value is 2450 MHz.
- As anticipated, filter loss at the passband edges (2300 MHz and 2400 MHz) is minimal. However, loss at the upper band edge is strongly correlated with the notch frequency; a lower notch frequency leads to increased loss.

Methodology

A multi-layer perceptron (MLP) is employed to map the filter's frequency response to its respective components. This decision is primarily based on the observation that existing literature regarding machine learning and artificial intelligence in circuit design tends to favor reinforcement learning (RL) for identifying the optimal circuit configuration. In this study, RL is applied following an initial estimate obtained through supervised learning regression. The advantage of utilizing an MLP lies in its structure as a neural network, which allows it to be incorporated into the RL framework as the initial value of the actor network.

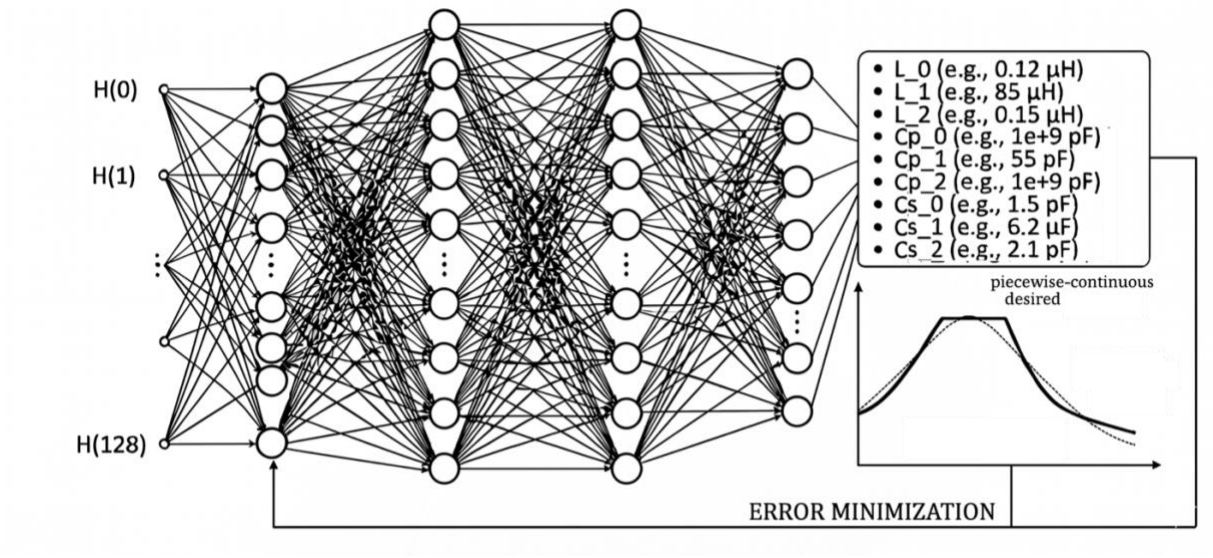

Figure 9: MLP

StandardScaler is used to normalize input features (frequencies), setting their mean to 0 and variance to 1 so that the neural network treats all frequency points equally during training. For output targets—capacitors (pF) and inductors (µH), which have very different scales—a log transform (np.log10) compresses their range. Next, a custom MixedScaler applies QuantileTransformer to components like Cs_0 and CS_1 with extreme outliers, while others receive standard scaling. After scaling, inputs from the critical passband (2300–2400 MHz) are multiplied by 100, ensuring the model focuses more closely and penalizes errors more heavily in this important region.

The MLP is implemented within the TunableMLP class in PyTorch. Rather than employing a fixed architecture, this class offers substantial flexibility by accepting parameters for input_dim, output_dim, hidden_layers, and neurons_per_layer. The network is constructed dynamically using an nn.Sequential container: it begins with an input layer, iteratively adds the specified number of hidden layers, and concludes with an output layer. Each hidden layer utilizes the ReLU (Rectified Linear Unit) activation function to introduce non-linearity. For regression tasks, the MLP applies Mean Squared Error (nn.MSELoss()), which is commonly used to minimize the squared discrepancies between predicted and actual values. The Adam optimizer is employed to adapt learning rates for each parameter, thereby enhancing the effectiveness of deep learning processes. Additionally, the MLP integrates the ReduceLROnPlateau scheduler, which monitors validation or test loss and automatically reduces the learning rate by half (factor=0.5) if no improvement is observed over ten epochs (patience=10). This feature facilitates more precise weight adjustment as the model approaches optimality, reducing the risk of overshooting.

The training cell operated efficiently, completing 100 epochs within approximately 41 seconds. The model converged effectively, with training MSE (Mean Squared Error) decreasing from around 28.5 to 0.19. Validation/test MSE exhibited similar performance, settling at approximately 0.198. The learning rate systematically declined from 0.0055 to 0.000345 during later epochs, illustrating the effectiveness of the ReduceLROnPlateau scheduler in taking progressively smaller steps near optimal weights to prevent overshooting. **No significant overfitting was observed:** The close alignment between final training loss and test loss indicates that the model achieved strong generalization rather than simply memorizing the training data.

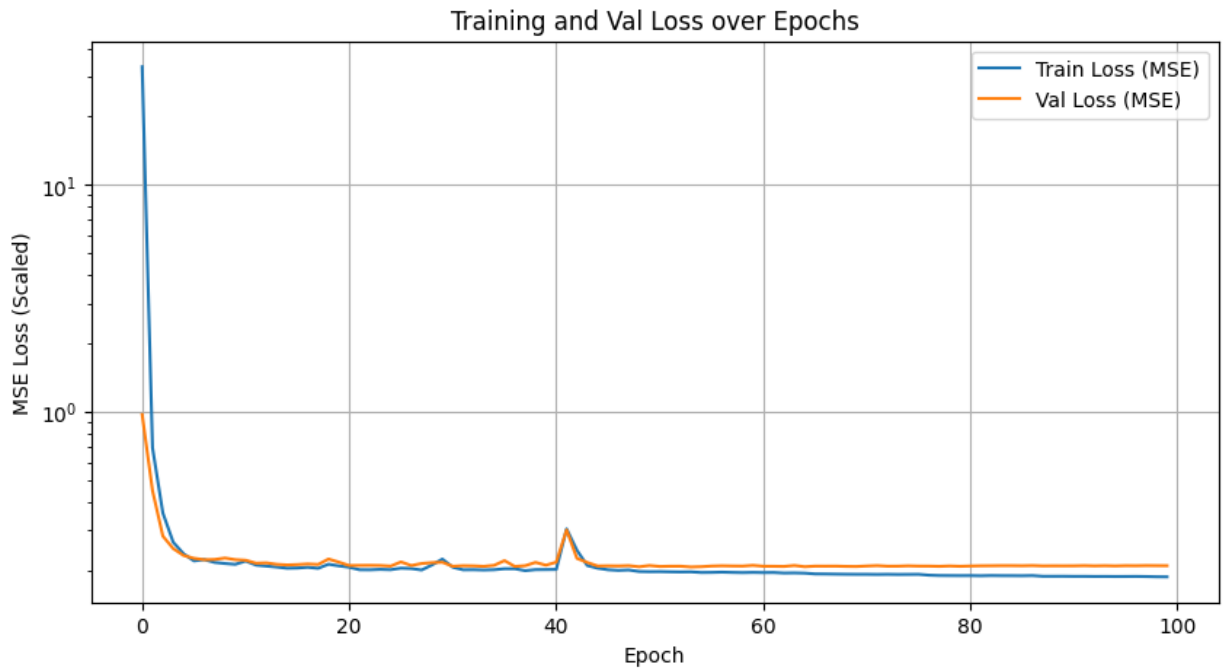

Figure 10: Training of MLP

For hyperparameter optimization, GridSearchCV from scikit-learn can be employed; however, several key considerations should be noted. GridSearchCV requires an estimator compatible with scikit-learn. Since an MLP model is a custom PyTorch neural network (nn.Module), it cannot be directly integrated into GridSearchCV. To enable compatibility, a wrapper library such as **skorch** is necessary, as it allows PyTorch models to function in a manner consistent with scikit-learn estimators. GridSearchCV conducts an exhaustive search, evaluating all defined parameter combinations. This approach can become extremely time-consuming and computationally intensive when applied to deep learning models with extensive search spaces involving parameters such as the number of hidden layers, neurons, batch sizes, and continuous variables like learning rates. In contrast, Optuna utilizes **Bayesian Optimization**, which leverages information from previous trials to predict promising hyperparameter values. This results in a more efficient and faster tuning process for neural networks compared to the brute-force methodology of grid search.

The optimization history graph illustrates how the model's error decreases over a series of trials, helping to identify which hyperparameters had the most impact. For complex tasks like filter synthesis, network capacity (such as neurons per layer and hidden layers) and learning rate often play the biggest roles in performance. The parallel coordinate plot offers even greater insight: each vertical line stands for a different hyperparameter (like learning rate or batch size), with the final line displaying the Objective Value. Every zigzagging path across the plot represents a single trial—a unique combination of hyperparameters—with lines color-coded by their objective value. Since the goal is to minimize MSE, pay attention to the dark blue lines (or the color that matches the lowest value on the color bar).

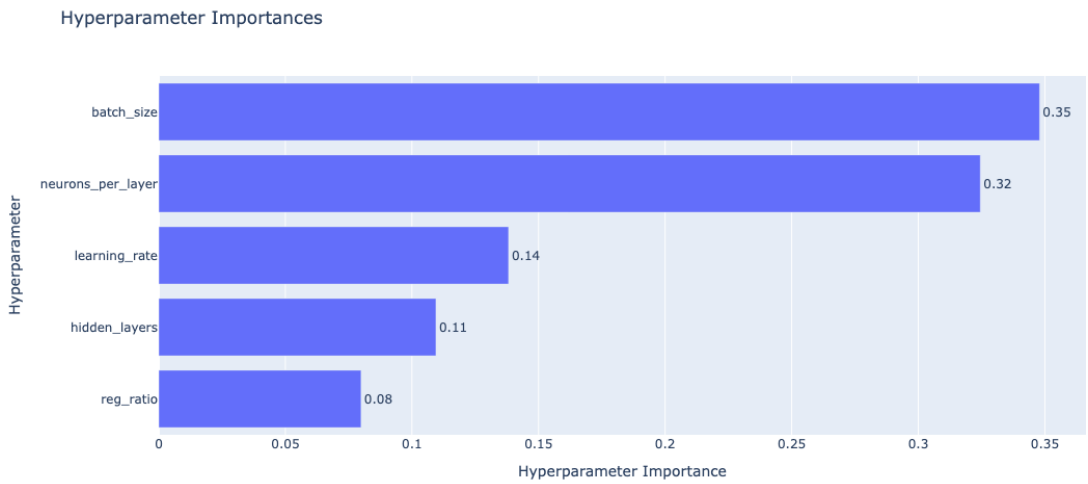

Figure 11: Importance of hyperparameters in reducing MSE

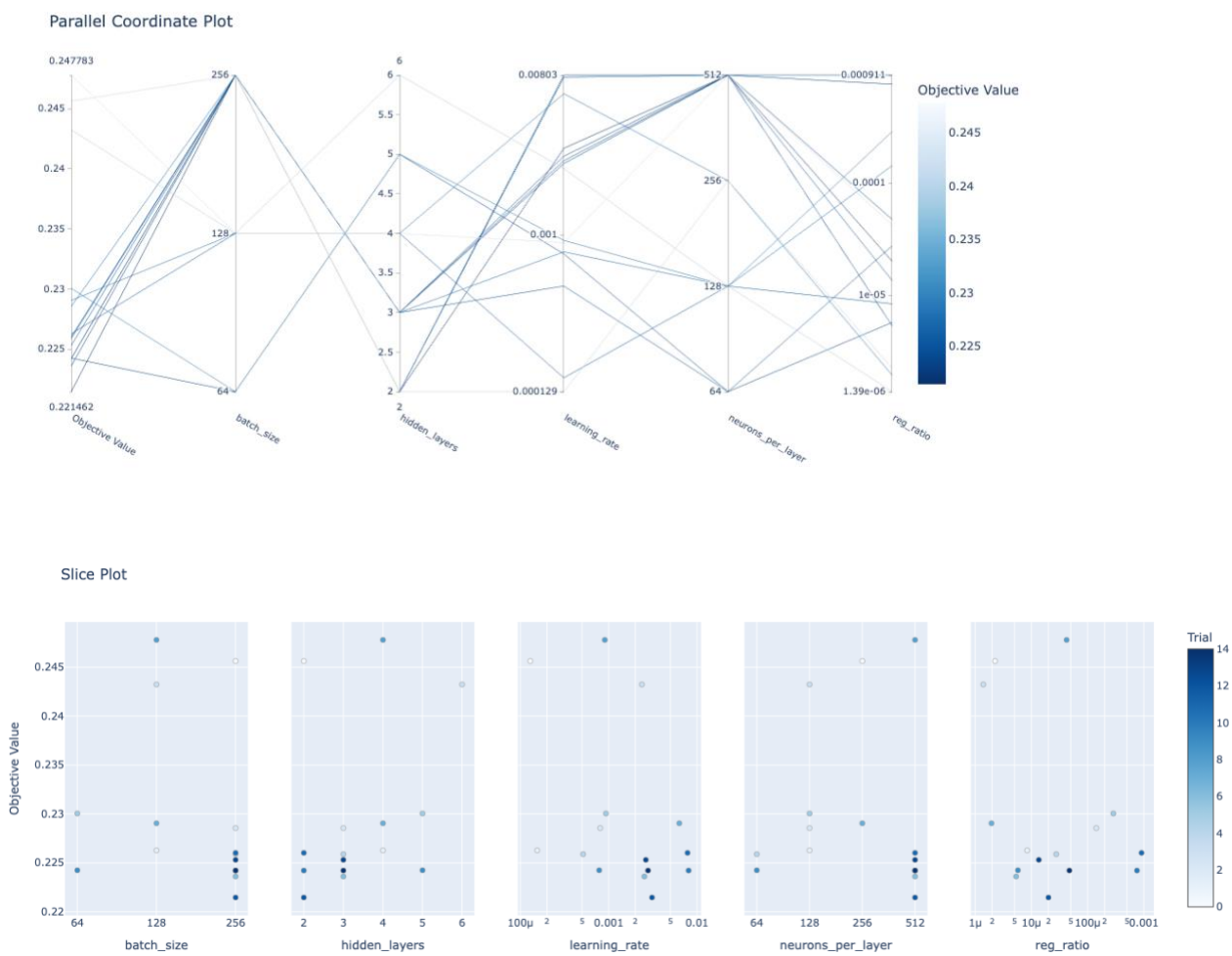

Figure 12: History of search for hyperparameter tuning

Results

Although MSE is useful for optimization, it lacks clear real-world meaning. To address this, the evaluation cell converts the scaled predictions back into physical units (micro-Henries and pico-Farads) and computes the Mean Absolute Percentage Error (MAPE). The average MAPE is 8.57% on training data and 8.93% on unseen test data, demonstrating a narrow margin and strong generalization. The breakdown table offers vital engineering insight: the model predicts inductors (L_0, L_1, L_2) and particular capacitors (such as Cs_1 and the Cp group) with impressive accuracy, often achieving errors below 1%. In contrast, it faces greater challenges with capacitors Cs_0 and Cs_2, which see around 33% error. This suggests that the frequency response is either highly sensitive or highly degenerate regarding these capacitor values, making them intrinsically more difficult to predict based solely on magnitude response.

```
Average Training Set MAPE (Component Values): 8.5727%
Average Test Set MAPE (Component Values): 8.9300%
```

	Component	Train MAPE (%)	Test MAPE (%)
0	L_0	0.361712	0.414147
1	L_1	0.030059	0.031772
2	L_2	0.371746	0.405734
3	Cs_0	33.822899	33.045464
4	Cs_1	0.031712	0.032686
5	Cs_2	33.392941	33.873947
6	Cp_0	1.255443	1.288161
7	Cp_1	6.623782	10.018766
8	Cp_2	1.263979	1.259364

Figure 13: MLP accuracy on predicting components values of filter

The MLP demonstrated strong performance when tested on a new piecewise continuous profile. Its initial Response MSE was **260.75**, and after applying Nelder-Mead optimization to adjust the circuit, the MSE improved only slightly to **254.91**. This minimal difference shows that the MLP's first prediction was already close to optimal. The component values had just an **8.54% MAPE** compared to those from full optimization, indicating that the model isn't simply memorizing its training data—it has genuinely absorbed the underlying physics and intricate mapping needed for filter synthesis. When confronted with a completely new, piecewise-continuous response, the MLP provides a nearly optimal solution in mere seconds.

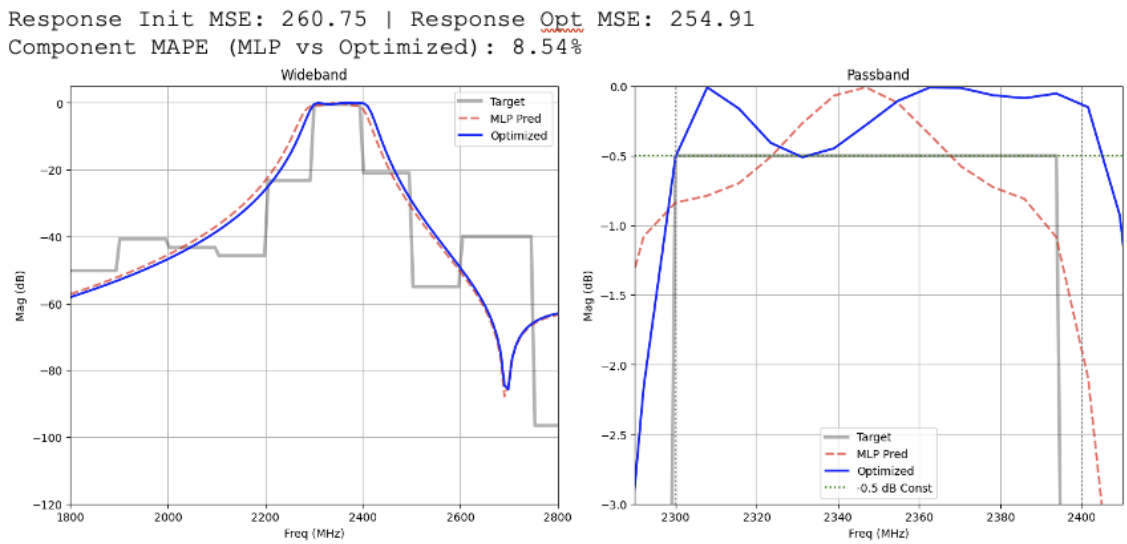

Figure 14: Comparison between MLP prediction (sample #159) and the corresponding optimal solutions

Filter design is inherently difficult for traditional linear models because the relationship between components (L ,C) and frequency response is highly non-linear. **MLP** and **SVR** (Support Vector Regression) are the right tools for this task because they excel at capturing these complex, high-dimensional mappings without requiring explicit physical equations for every iteration.

The details of deployed SVR model is not discussed here but workflow is identical to MLP. The notebook supplied to support the comparison table shown below.

The data confirms that both models are roughly equal in performance, offering nearly interchangeable results. While SVR shows a slight edge in training MAPE (7.69% vs. 8.57%), the MLP actually delivers a better initial frequency response out of the box, with an average MSE of 597.83 compared to SVR's 620.06. Once these predictions are put through a targeted optimization routine, both settle to very close average MSE of 532 and 523.

Even on individual test cases like Sample #159, the parity remains clear. The MLP achieved a tighter component match (8.54% MAPE vs. 9.27% for SVR), while SVR reached a marginally lower optimized error (244.53 vs. 254.91). This statistical neck-and-neck performance proves that both architectures have successfully mastered the underlying design physics, allowing you to choose the model that best fits your specific computational workflow without sacrificing accuracy.

	MLP	SVR
Training MAPE for filter components	8.57%	7.69%
Test MAPE for filter components	8.93%	7.94%
Frequency Response MSE (Averaged over 100 cases)	597.83	620.06
Optimized Frequency Response MSE (Averaged over 100 cases)	532.66	523.06
Sample #159 MSE	260.75	261.50
Optimized sample #159 MSE	254.91	244.53
Component MAPE for sample #159	8.54%	9.27%

Figure 15:Comparison table between MLP and SVR

Next steps

In deep learning workflows, supervised learning (SL**)** is often the first line of defense for regression problems, providing a "hot start" by mapping inputs to preliminary solutions. In the context of LC filter synthesis, architectures like MLP and SVM are used for this project.

However, while these models successfully learn the underlying physics and provide solutions with low MAPE (around 8.5%), the search space for high-frequency electronics is exceptionally sensitive. Because the relationship between a component's physical value and its frequency response highly non-linear, even a 1% deviation in an inductor or capacitor value can cause a filter to drift out of specification. In these high-precision domains, a supervised "one-shot" prediction often acts as an educated guess—valuable, but inadequate for a realizable final product.

To bridge this gap, Reinforcement Learning (RL)—specifically algorithms like Proximal Policy Optimization (PPO) via Stable Baselines3—has emerged as a powerful tool for navigating these massive solution spaces. Much like RL agents learn to master the branching paths of Go or the complex controls of autonomous vehicles, they can be trained to "play" the design space of an electrical network.

By combining these paradigms, we could establish a two-stage optimization pipeline:

- **The Supervised Estimate:** The MLP/SVR provides a near-optimal starting point in a fraction of a second, drastically reducing the computational overhead that an iterative search would normally face from a "cold start."
- **The RL Refinement:** The RL agent (or a targeted routine like Nelder-Mead) then takes this estimate and performs a granular search of the immediate local space. It fine-tunes the component values to minimize the response MSE, turning a "degraded" prediction into a continuous, realizable frequency response.

This synergy allows for a design tool that is both fast and precise, replacing manual, ad-hoc tuning with an automated system that understands the deep, non-linear mapping of RF components.

Outline of project

Project is structured in three Colab notebooks. These are standalone notebooks.

1- The project shall start with execution of data generation and clean up notebook.

Data generation notebook:

https://github.com/shayanfarahvash/CAL_Capstone_Project/blob/dd40ec38293e1b441bdf9251332f4030ca58ef34/Filter_data.ipynb

This data generates three data files that are used later by notebooks for implementation of SVR and MLP. All file read/write operations in these notebooks are done through Google drive. The code must be changed to avoid access errors.

2- The operation of MLP notebook is described in detail in this document. Notebook can be found here:

MLP Notebook:

https://github.com/shayanfarahvash/CAL_Capstone_Project/blob/dd40ec38293e1b441bdf9251332f4030ca58ef34/Filter_MLP.ipynb

3- As said previously the SVR notebook execution flow is identical to MLP. It is recommended that MLP to be executed first. After gaining familiarity with code execution flow, running SVR notebook would be far easier. SVR notebook can be fund here:

SVR Notebook:

https://github.com/shayanfarahvash/CAL_Capstone_Project/blob/dd40ec38293e1b441bdf9251332f4030ca58ef34/Filter_SVR.ipynb